

CHAPTER 3

BOOLEAN ALGEBRA

3.1 Introduction

As pointed out in Chapter 1, digital logic circuits operate in the binary mode, where each input and output is either 0 or 1. This characteristic of logic circuits allows us to use *Boolean algebra* as a tool for analysis and design of digital systems. In this chapter we shall discuss Boolean algebra in details.

3.2 Axiomatic Definition of Boolean Algebra

Boolean algebra, like any other deductive mathematical system, may be defined with a set of elements, a set of operators, and a number of unproved axioms or postulates. A *set* of elements is any collection of objects having a common property. If S is a set, and x and y are certain objects, then $x \in S$ denotes that x is a member of the set S , and $y \notin S$ denotes that y is not an element of S . A set with a denumerable number of elements is specified by braces: $A = \{1, 2, 3, 4\}$, i.e., the elements of set A are the numbers 1, 2, 3, and 4. A *binary operator* defined on a set S of elements is a rule that assigns to each pair of elements from S a unique element from S . As an example, consider the relation $a * b = c$. We say that $*$ is a binary operator if it specifies a rule for finding c from the pair (a, b) and also if $a, b, c \in S$. However, $*$ is not a binary operator if $a, b \in S$, while the rule finds $c \notin S$. The *postulates* of a mathematical system form the basic assumptions from which it is possible to deduce the rules, theorems, and properties of the system.

In 1854 George Boole introduced a systematic treatment of logic and developed for this purpose an algebraic system now called *Boolean algebra*. In 1938 C. E. Shannon introduced a two-valued Boolean algebra called *switching algebra*, in which he demonstrated that the properties of bistable electrical switching circuits could be represented by this algebra. For the formal definition of Boolean algebra, we shall employ the postulates formulated by E. V. Huntington in 1904.

3.2.1 Postulates of Boolean Algebra

Boolean algebra is an algebraic structure defined on a set of elements B together with two binary operators $+$ and $\cdot$ provided the following (Huntington) postulates are satisfied:

1. (a) Closure with respect to the operator $+$: For $x, y \in B$, $x + y = c \in B$.
(b) Closure with respect to the operator $\cdot$: For $x, y \in B$, $x \cdot y = c \in B$.
2. (a) An identity element with respect to $+$, designated by 0: For $x \in B$, $x + 0 = 0 + x = x$.
(b) An identity element with respect to $\cdot$, designated by 1: For $x \in B$, $x \cdot 1 = 1 \cdot x = x$.

3. (a) Commutative with respect to $+$: For $x, y \in B$, $x + y = y + x$.
 (b) Commutative with respect to $\cdot$: For $x, y \in B$, $x \cdot y = y \cdot x$.
4. (a) $\cdot$ is distributive over $+$: For $x, y, z \in B$, $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$.
 (b) $+$ is distributive over $\cdot$: For $x, y, z \in B$, $x + (y \cdot z) = (x + y) \cdot (x + z)$.
5. For every element $x \in B$, there exists an element $x' \in B$ (called the *complement* of x) such that:
 - (a) $x + x' = 1$, and
 - (b) $x \cdot x' = 0$.
6. There exists at least two elements $x, y \in B$ such that $x \neq y$.

It is important to distinguish between the elements of the set of an algebraic structure and the variables of an algebraic system. In Boolean algebra, one defines the elements of the set B , and variables such as x, y, z are merely symbols that represent the elements. At this point, it is important to realize that in order to have a Boolean algebra, one must show:

1. the elements of the set B ,
2. the rules of operation for the two binary operators, and
3. that the set of elements B , together with the two operators, satisfies the six Huntington postulates.

One can formulate many Boolean algebra, depending on the choice of elements of B and the rules of operation. In this book, we deal only with a two-valued Boolean algebra, i.e., one with only two elements.

3.2.2 Two-valued Boolean Algebra

A two-valued Boolean algebra is defined on a set of two elements, $B = \{0, 1\}$, with rules for the two binary operators $+$ and $\cdot$ as shown in Table 3.1 (the rule for the complement operator is for verification of postulate 5):

Table 3.1 Two-valued Boolean operator tables.

$x \ y$	$x \cdot y$
0 0	0
0 1	0
1 0	0
1 1	1

$x \ y$	$x + y$
0 0	0
0 1	1
1 0	1
1 1	1

x	x'
0	1
1	0

These rules are exactly the same as the AND, OR, and NOT operations, respectively, defined in Table 1.1. We must now show that the Huntington postulates are valid for the set $B = \{0, 1\}$ and the two binary operators defined in Table 3.1.

1. *Closure* is obvious from the two binary operators defined in Table 3.1, since the result of each operation is either 0 or 1 and $0, 1 \in B$.

2. From the two binary operators of Table 3.1, we see that

(a) $0 + 0 = 0$ and $0 + 1 = 1 + 0 = 1$

(b) $1 \cdot 1 = 1$ and $1 \cdot 0 = 0 \cdot 1 = 0$

which establishes the two *identity elements* 0 for + and 1 for $\cdot$ as defined by postulate 2.

3. The *commutative* laws are obvious from the symmetry of the binary operators in Table 3.1.

4. (a) The *distributive law* $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$ can be shown to hold true from the binary operators of Table 3.1 by forming a truth table of all possible values of x , y , and z . For each combination, we derive $x \cdot (y + z)$ and show that the value is the same as $(x \cdot y) + (x \cdot z)$, as shown in Table 3.2.

Table 3.2 Proof of the distributive law $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$

$x \ y \ z$	$y + z$	$x \cdot (y + z)$	$x \cdot y$	$x \cdot z$	$(x \cdot y) + (x \cdot z)$
0 0 0	0	0	0	0	0
0 0 1	1	0	0	0	0
0 1 0	1	0	0	0	0
0 1 1	1	0	0	0	0
1 0 0	0	0	0	0	0
1 0 1	1	1	0	1	1
1 1 0	1	1	1	0	1
1 1 1	1	1	1	1	1

(b) The *distributive law* $x + (y \cdot z) = (x + y) \cdot (x + z)$ can be shown to hold true from the binary operators of Table 3.1 by forming a truth table of all possible values of x , y , and z . For each combination, we derive $x + (y \cdot z)$ and show that the value is the same as $(x + y) \cdot (x + z)$, as shown in Table 3.3.

Table 3.3 Proof of the distributive law $x + (y \cdot z) = (x + y) \cdot (x + z)$

$x \ y \ z$	$y \cdot z$	$x + (y \cdot z)$	$x + y$	$x + z$	$(x + y) \cdot (x + z)$
0 0 0	0	0	0	0	0
0 0 1	0	0	0	1	0
0 1 0	0	0	1	0	0
0 1 1	1	1	1	1	1
1 0 0	0	1	1	1	1
1 0 1	0	1	1	1	1
1 1 0	0	1	1	1	1
1 1 1	1	1	1	1	1

5. From the complement operator in Table 3.1, it is easily shown that:
 - (a) $x + x' = 1$, since $0 + 0' = 0 + 1 = 1$ and $1 + 1' = 1 + 0 = 1$.
 - (b) $x \cdot x' = 0$, since $0 \cdot 0' = 0 \cdot 1 = 0$ and $1 \cdot 1' = 1 \cdot 0 = 0$.
 which verifies postulate 5.
6. Postulate 6 is satisfied because the two-valued Boolean algebra has two distinct elements 0 and 1 with $0 \neq 1$.

We have just established a two-valued Boolean algebra having a set of two elements, 0 and 1, two binary operators with operation rules equivalent to the AND and OR operations, and a complement operation equivalent to the NOT operation. From here on, we shall drop the adjective "two-valued" from Boolean algebra in subsequent discussion.

3.3 Basic Theorems and Properties of Boolean Algebra

3.3.1 Duality Principle

The Huntington postulates have been listed in pairs and designated by part (a) and part (b). One part may be obtained from the other if the binary operators and the identity elements are interchanged. This important property of Boolean algebra is called *duality principle*. It states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. In a two-valued Boolean algebra, the identity elements and the elements of the set B are the same, 0 and 1. The duality principle has many applications. If the *dual* of an algebraic expression is desired, we simply interchange OR and AND operators and replace 0s by 1s and 1s by 0s.

3.3.2 Basic Theorems of Boolean Algebra

Table 3.4 lists six theorems of Boolean algebra and four of its postulates. The notation is simplified by omitting the $\cdot$ whenever this does not lead to confusion. The theorems and postulates listed are the most basic relationships in Boolean algebra. The theorems, like the postulates, are listed in pairs; each relation is the dual of the one paired with it. The postulates are basic axioms of the algebraic structure and need no proof. The theorems must be proved from the postulates.

Table 3.4 Postulates and theorems of Boolean algebra.

Postulate 2	(a) $x + 0 = x$	(b) $x \cdot 1 = x$
Postulate 3 (<i>commutative law</i>)	(a) $x + y = y + x$	(b) $xy = yx$
Postulate 4 (<i>distributive law</i>)	(a) $x(y + z) = xy + xz$	(b) $x + yz = (x + y)(x + z)$
Postulate 5	(a) $x + x' = 1$	(b) $x \cdot x' = 0$
Theorem 1	(a) $x + x = x$	(b) $x \cdot x = x$
Theorem 2	(a) $x + 1 = 1$	(b) $x \cdot 0 = 0$
Theorem 3 (<i>involution law</i>)	$(x')' = x$	
Theorem 4 (<i>associative law</i>)	(a) $x + (y + z) = (x + y) + z$	(b) $x(yz) = (xy)z$
Theorem 5 (<i>DeMorgan's theorem</i>)	(a) $(x + y)' = x'y'$	(b) $(xy)' = x' + y'$
Theorem 6 (<i>absorption law</i>)	(a) $x + xy = x$	(b) $x(x + y) = x$

The proofs of the theorems are presented below. At the right is listed the number of the postulates and previously proved theorems which justifies each step of the proof.

Theorem 1(a): $x + x = x$.

$$\begin{aligned}
 x + x &= (x + x) \cdot 1 && \text{by postulate:} && 2(b) \\
 &= (x + x)(x + x') && && 5(a) \\
 &= x + xx' && && 4(b) \\
 &= x + 0 && && 5(b) \\
 &= x && && 2(a)
 \end{aligned}$$

Theorem 1(b): $x \cdot x = x$.

$$\begin{aligned}
 x \cdot x &= xx + 0 && \text{by postulate:} && 2(a) \\
 &= xx + xx' && && 5(b) \\
 &= x(x + x') && && 4(a) \\
 &= x \cdot 1 && && 5(a) \\
 &= x && && 2(b)
 \end{aligned}$$

Note that theorem 1(b) is the dual of theorem 1(a) and that each step of the proof in part (b) is the dual of part (a). Any dual theorem can be similarly derived from the proof of its corresponding pair.

Theorem 2(a): $x + 1 = 1$.

$$\begin{aligned}
 x + 1 &= (x + 1) \cdot 1 && \text{by postulate:} && 2(b) \\
 &= 1 \cdot (x + 1) && && 3(b) \\
 &= (x + x')(x + 1) && && 5(a) \\
 &= x + x' \cdot 1 && && 4(b) \\
 &= x + x' && && 2(b) \\
 &= 1 && && 5(a)
 \end{aligned}$$

Theorem 2(b): $x \cdot 0 = 0$ by duality.

Theorem 3: $(x')' = x$.

From postulate 5, we have $x + x' = 1$ and $x \cdot x' = 0$, which defines the complement of x as x' . Therefore, the complement of x' is x , that is, $(x')'$ is x .

Theorem 4(a): $x + (y + z) = (x + y) + z$.

$$\begin{aligned}
 x + (y + z) &= x + (y + z) \cdot 1 && \text{by postulate:} && 2(b) \\
 &= x + 1 \cdot (y + z) && && 3(b) \\
 &= x + 1 \cdot y + 1 \cdot z && && 4(a) \\
 &= x + y \cdot 1 + z \cdot 1 && && 3(b) \\
 &= x \cdot 1 + y \cdot 1 + z && && 2(b) \\
 &= 1 \cdot x + 1 \cdot y + z && && 3(b) \\
 &= 1 \cdot (x + y) + z && && 4(a) \\
 &= (x + y) \cdot 1 + z && && 3(b) \\
 &= (x + y) + z && && 2(b)
 \end{aligned}$$

Theorem 4(b): $x(yz) = (xy)z$ by duality.

Theorem 5(a): $(x + y)' = x'y'$.

We are to prove that the complement of $(x + y)$ is $x'y'$. From postulate 5, we have to prove $(x + y) + x'y' = 1$ and $(x + y) \cdot x'y' = 0$.

$$\begin{aligned}
 (x + y) + x'y' &= x + (y + x'y') && \text{by theorem} && 4(a) \\
 &= x + (y + x')(y + y') && \text{by postulate} && 4(b) \\
 &= x + (y + x') \cdot 1 && \text{by postulate} && 5(a)
 \end{aligned}$$

$= x + 1 \cdot (y + x')$	by postulate	3(b)
$= x + 1 \cdot y + 1 \cdot x'$	by postulate	4(a)
$= x + y \cdot 1 + x' \cdot 1$	by postulate	3(b)
$= x + y + x'$	by postulate	2(b)
$= y + x + x'$	by postulate	3(a)
$= y + 1$	by postulate	5(a)
$= 1$	by theorem	2(a)

$(x + y) \cdot x'y' = x'y' \cdot (x + y)$	by postulate	3(b)
$= x'y'x + x'y'y$	by postulate	4(a)
$= y'x'x + x'yy'$	by postulate	3(b)
$= y'xx' + x'yy'$	by postulate	3(b)
$= y' \cdot 0 + x' \cdot 0$	by postulate	5(b)
$= 0 + 0$	by theorem	2(b)
$= 0$	by theorem	1(a)

Theorem 5(b): $(xy)' = x' + y'$ by duality.

Theorem 6(a): $x + xy = x$.

$x + xy = x \cdot 1 + xy$	by postulate	2(b)
$= x(1 + y)$	by postulate	4(a)
$= x(y + 1)$	by postulate	3(a)
$= x \cdot 1$	by theorem	2(a)
$= x$	by postulate	2(b)

Theorem 6(b): $x(x + y) = x$ by duality.

DeMorgan's Theorem for More Than Two Variables

DeMorgan's theorem can be extended to three or more variables. The three-variable form of the first DeMorgan's theorem is derived below.

$(A + B + C)' = (A + X)'$	let $B + C = X$
$= A'X'$	by theorem 5(a)
$= A' \cdot (B + C)'$	substituting $B + C = X$
$= A' \cdot (B'C')$	by theorem 5(a)
$= A'B'C'$	by theorem 4(b)

$(ABC)' = A' + B' + C'$ by duality.

DeMorgan's theorem for any number of variables resemble in form the two-variable case and can be derived by successive substitution similar to the method used in the above derivation. These theorems can be generalized as follows:

$$(A + B + C + D + E + \dots)' = A'B'C'D'E'\dots$$

$$(ABCDE\dots)' = A' + B' + C' + D' + E' + \dots$$

3.3.3 Operator Precedence

The operator precedence for evaluating Boolean expression is (1) parentheses, (2) NOT, (3) AND, and (4) OR. In other words, the expression inside the parentheses must be evaluated before all other operations. The next operation that holds precedence is the NOT, then follows the AND, and finally the OR. For example, the evaluation of the Boolean expression $xy' + z(x + y)$ for $x = 0$, $y = 1$, and $z = 0$ is shown below:

$$\begin{aligned} xy' + z(x + y) &= 0 \cdot 1' + 0 \cdot (0 + 1) \\ &= 0 \cdot 1' + 0 \cdot 1 && \text{evaluation of parenthesis} \\ &= 0 \cdot 0 + 0 \cdot 1 && \text{evaluation of NOT} \\ &= 0 + 0 && \text{evaluation of AND} \\ &= 0 && \text{evaluation of OR} \end{aligned}$$

3.4 Boolean Function

3.4.1 Algebraic Expression of Boolean Function

A binary variable can take the value of either 0 or 1. A *Boolean function* is an expression formed with binary variables, the two binary operators OR and AND, the unary operator NOT, parentheses, and equal sign. For a given value of the variables, the functional value of the expression can be either 0 or 1. The following is an algebraic expression of a Boolean function:

$$F_1 = xyz'$$

The function F_1 is equal to 1 if $x = 1$ and $y = 1$ and $z = 0$; otherwise $F_1 = 0$.

3.4.2 Truth Table Representation of Boolean Function

A Boolean function may be represented in a truth table. To represent a Boolean function in a truth table, we need a list of the 2^n combinations of 0 and 1 of the n binary variables, and a column showing the functional value of either 0 or 1 for each combination of values of the n variables. As shown in Table 3.5, there are eight possible distinct combinations for assigning bits of three variables. The column labeled F_1 contains either a 0 or a 1 for each of these combinations. The table shows that the function F_1 is equal to 1 only when $x = 1$, $y = 1$, and $z = 0$. It is equal to 0 otherwise. Consider now the function:

$$F_2 = x + y'z$$

The function $F_2 = 1$ if $x = 1$; or $y = 0$ and $z = 1$. In Table 3.5, $x = 1$ in the last four rows; $y = 0$ and $z = 1$ in rows 001 and 101. The latter combination applies also for $x = 1$. Therefore, there are five combinations that make $F_2 = 1$.

As a third example, consider the function:

$$F_3 = x'y'z + x'yz + xy'$$

The function $F_3 = 1$ if $x = 0$ and $y = 0$ and $z = 1$; or $x = 0$ and $y = 1$ and $z = 1$; or $x = 1$ and $y = 0$. $x = 1$ and $y = 0$ in rows 100 and 101. This is shown in Table 3.5 with four 1s and four 0s.

As a fourth example, consider the function:

$$F_4 = xy' + x'z$$

The function $F_4 = 1$ if $x = 1$ and $y = 0$; or $x = 0$ and $z = 1$. $x = 1$ and $y = 0$ in rows 100 and 101. $x = 0$ and $z = 1$ in rows 001 and 011. This is shown in Table 3.5 with four 1s and four 0s.

Table 3.5 Truth tables for $F_1 = xyz'$, $F_2 = x + y'z$,
 $F_3 = x'y'z + x'yz + xy'$, and $F_4 = xy' + x'z$

$x y z$	F_1	F_2	F_3	F_4
0 0 0	0	0	0	0
0 0 1	0	1	1	1
0 1 0	0	0	0	0
0 1 1	0	0	1	1
1 0 0	0	1	1	1
1 0 1	0	1	1	1
1 1 0	1	1	0	0
1 1 1	0	1	0	0

3.4.3 Non-unique Algebraic Expression of Boolean Function

From Table 3.5, we find that F_3 is equal to F_4 , since both have identical 0s and 1s for each combination of values of the three binary variables. In general, two functions of n binary variables are said to be equal if they have the same value for all possible 2^n combinations of the n variables. Though F_3 is equal to F_4 , the algebraic expressions for F_3 and F_4 are not same. Therefore, the algebraic expression of a given Boolean function is not unique. In other words, it is possible to find more than one algebraic expression that specify the same Boolean function. As

a matter of fact, the manipulation of Boolean algebra is applied mostly to the problem of finding simpler expression for the same function. The *simplification* of F_3 to F_4 can be achieved as below:

$$\begin{aligned}
 F_3 &= x'y'z + x'yz + xy' \\
 &= x'zy' + x'zy + xy' && \text{by postulate:} && 3(b) \\
 &= x'z(y' + y) + xy' && 4(a) \\
 &= x'z(y + y') + xy' && 3(a) \\
 &= x'z \cdot 1 + xy' && 5(a) \\
 &= x'z + xy' && 2(b) \\
 &= xy' + x'z && 3(a) \\
 &= F_4
 \end{aligned}$$

Simplification of Boolean function will be discussed in detail in Chapter 5.

3.4.4 Logic Diagram

A Boolean function may be transformed from an algebraic expression into a *logic diagram* composed of AND, OR, and NOT gates. The implementation of the four Boolean functions introduced in the previous discussion is shown in Figure 3.1. The logic diagram includes a NOT gate for every variable present in its complement form. There is an AND gate for each AND term in the expression, and an OR gate is used to combine two or more AND terms. From the diagram, it is obvious that the implementation of F_4 requires fewer gates and fewer inputs than F_3 . Since F_3 and F_4 are equal Boolean functions, it is more economical to implement the F_4 form than the F_3 form. To find simpler circuits, one must know how to manipulate Boolean functions to obtain equal and simpler expressions.

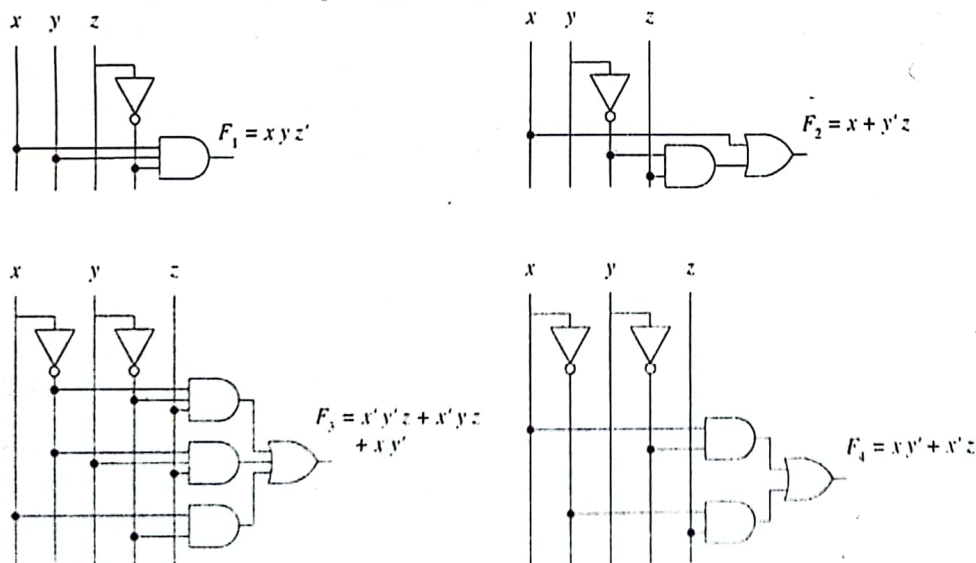


Figure 3.1 Implementation of Boolean functions with gates.

3.4.5 Complement of Boolean Function

The complement of a Boolean function F is F' and is obtained from an interchange of 0s for 1s and 1s for 0s in the value of F . The complement of a Boolean function may be derived algebraically through DeMorgan's theorem. For example, the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$ can be found as follows:

$$\begin{aligned}
 F_1' &= (x'yz' + x'y'z)' \\
 &= (x'yz')'(x'y'z)' \\
 &= (x + y' + z)(x + y + z') \\
 F_2' &= (x(y'z' + yz))' \\
 &= x' + (y'z' + yz)' \\
 &= x' + (y'z')'(yz)' \\
 &= x' + (y + z)(y' + z')
 \end{aligned}$$

3.5 Forms of Boolean Functions

3.5.1 Minterms and Maxterms

A binary variable may appear either in its normal form (x) or in its complement form (x'). Now consider two binary variables x and y combined with an AND operation. Since each variable may appear in either form, there are four possible combinations: $x'y'$, $x'y$, xy' , and xy . Each of these four AND terms is called a *minterm* or a *standard product term* (or *standard product*). In a similar manner, n variables can be combined to form 2^n minterms. The 2^n different minterms may be determined by a method similar to the one shown in Table 3.6 for three variables. The binary numbers from 0 to $2^n - 1$ are listed under the n variables. Each minterm is obtained from an AND term of the n variables, with each variable being primed (or complemented) if the corresponding bit of the binary number is a 0 and unprimed (or normal) if a 1. A symbol for each minterm is also shown in the table and is of the form m_j , where j denotes the decimal equivalent of the binary number of the minterm designated.

In a similar fashion, n variables forming an OR term, with each variable being primed or unprimed, provide 2^n possible combinations, called *maxterms* or *standard sum terms* (or *standard sums*). The eight maxterms for three variables, together with their symbolic designation, are listed in Table 3.6. Any 2^n maxterms for n variables may be determined similarly. Each maxterm is obtained from an OR term of the n variables, with each variable being unprimed if the corresponding bit is 0 and primed if a 1. Note that *each maxterm is the complement of its corresponding minterm and vice versa*.

Table 3.6 Minterm and maxterm for three binary variables.

Variables $x \ y \ z$	Minterms		Maxterms	
	Term	Designation	Term	Designation
0 0 0	$x'y'z'$	m_0	$x + y + z$	M_0
0 0 1	$x'y'z$	m_1	$x + y + z'$	M_1
0 1 0	$x'yz'$	m_2	$x + y' + z$	M_2
0 1 1	$x'yz$	m_3	$x + y' + z'$	M_3
1 0 0	$xy'z'$	m_4	$x' + y + z$	M_4
1 0 1	$xy'z$	m_5	$x' + y + z'$	M_5
1 1 0	xyz'	m_6	$x' + y' + z$	M_6
1 1 1	xyz	m_7	$x' + y' + z'$	M_7

3.5.2 Canonical Forms

Boolean functions expressed in terms of minterms or maxterms are said to be in canonical form, because these representations are unique for the given functions. There are two canonical forms of Boolean functions: *canonical sum of products* (CSOP) and *canonical product of sums* (CPOS).

Canonical Sum of Products

A Boolean function may be expressed algebraically from a given truth table by forming a minterm for each combination of the variables which produces a 1 in the function, and then taking the OR of all these terms. For example, the function f_1 in Table 3.7 is determined by expressing the combinations 001, 100, and 111 as $x'y'z$, $xy'z'$, and xyz , respectively. Since each one of these minterms results in $f_1 = 1$, we should have:

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7.$$

Similarly, it may be easily verified that:

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7.$$

Table 3.7 Arbitrary functions of three variables.

$x \ y \ z$	Function f_1	Function f_2
0 0 0	0	0
0 0 1	1	0
0 1 0	0	0
0 1 1	0	1
1 0 0	1	0
1 0 1	0	1
1 1 0	0	1
1 1 1	1	1

These examples demonstrate an important property of Boolean algebra: *Any Boolean function can be expressed as a sum of minterms* (by “sum” is meant the ORing of terms). As these expressions are unique for the given Boolean functions, they are called *canonical sum of products (CSOP)* expressions. They are also called *sum of minterms* expressions.

Short Notation of Canonical Sum of Products Function

It is sometimes convenient to express the Boolean function, when in its canonical sum of products form, in the following short notation:

$$f_1(x, y, z) = \sum(1, 4, 7)$$

$$f_2(x, y, z) = \sum(3, 5, 6, 7)$$

The summation symbol, $\sum$, stands for the ORing of terms; the numbers following it are the minterms of the function. The letters in parenthesis following f_1 or f_2 form a list of the variables in the order taken when the minterm is converted to an AND term.

For n binary variables, one can obtain 2^n distinct minterms, and any Boolean function can be expressed as a sum of minterms. The minterms whose sum defines the Boolean function are those that give the 1s of the function in a truth table. Since the function can be either 0 or 1 for each minterm, and since there are 2^n minterms, one can calculate the number of possible functions that can be formed with n variables to be 2^{2^n} .

Canonical Product of Sums

Now consider the complement of a Boolean function. It may be read from the truth table by forming a minterm for each combination that produces a 0 in the function and then ORing those terms. From Table 3.7, the complement of f_1 is read as:

$$f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

If we take the complement of f_1' , we obtain the function f_1 :

$$\begin{aligned} f_1 &= (x'y'z' + x'yz' + x'yz + xy'z + xyz')' \\ &= (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z')(x' + y' + z) \\ &= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 \end{aligned}$$

Similarly, from Table 3.7, it is possible to read the expression for f_2 :

$$\begin{aligned} f_2 &= (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) \\ &= M_0 \cdot M_1 \cdot M_2 \cdot M_4 \end{aligned}$$

These examples demonstrate a second important property of Boolean algebra: *Any Boolean function can be expressed as a product of maxterms* (by "product" is meant the ANDing of terms). The procedure for obtaining the product of maxterms directly from the truth table is as follows. Form a maxterm for each combination of the variables which produces a 0 in the function, and then form the AND of all these maxterms. Each of the 2^n functions of n binary variables can be expressed as a product of maxterms.

As these expressions are also unique for the given Boolean functions, they are called *canonical product of sums (CPOS)* expression. They are also called *product of maxterms* expression.

Short Notation of Canonical Product of Sums Function

A convenient way to express canonical product of sums expression is as follows:

$$f_1(x, y, z) = \prod (0, 2, 3, 5, 6)$$

$$f_2(x, y, z) = \prod (0, 1, 2, 4)$$

The product symbol, $\prod$, denotes the ANDing of maxterms; the number following it are the maxterms of the function.

Conversion between Canonical Forms

The complement of a function expressed as the canonical sum of products equals the sum of minterms missing from the original function. This is because the original function is expressed by those minterms that make the function equal to 1, while its complement is a 1 for those minterms that the function is a 0. As an example, consider the function:

$$F(A, B, C) = \sum (1, 4, 5, 6, 7)$$

This has a complement that can be expressed as:

$$F'(A, B, C) = \sum (0, 2, 3) = m_0 + m_2 + m_3$$

Now, if we take the complement of F' by DeMorgan's theorem, we obtain F in a different form:

$$\begin{aligned} F(A, B, C) &= (m_0 + m_2 + m_3)' = m_0' \cdot m_2' \cdot m_3' = M_0 \cdot M_2 \cdot M_3 \\ &= \prod (0, 2, 3) \end{aligned}$$

The last conversion follows from the definition of minterms and maxterms as shown in Table 3.6. From the table, it is clear that the following relation holds true:

$$m_j' = M_j$$

That is, the maxterm with subscript j is a complement of the minterm with the same subscript j , and vice versa.

The last example demonstrates the conversion between a function expressed in canonical sum of products and its equivalent in canonical product of sums. A similar argument will show that the conversion between the canonical product of sums and the canonical sum of products is similar. We now state a general conversion procedure. To convert from one canonical form to another, interchange the symbols $\sum$ and $\prod$ and list those numbers missing from the original form. As another example, the function:

$$F(x, y, z) = \prod(0, 2, 4, 5)$$

is expressed in the canonical product of sums form. Its conversion to canonical sum of products is:

$$F(x, y, z) = \sum(1, 3, 6, 7)$$

Note that, in order to find the missing terms, one must realize that the total number of minterms or maxterm is 2^n , where n is the number of binary variables in the function.

3.5.3 Noncanonical Forms

The two canonical forms of Boolean algebra are basic forms that one obtains from reading a function from the truth table. In these forms, each minterm or maxterm contains n literals, where n is the number of variables. A *literal* is a primed (complemented) or unprimed (uncomplemented or normal) variable.

Another way to express Boolean functions is in *noncanonical form*. In this configuration, the terms that form the function may contain one, two or any number of literals. There are two types of noncanonical forms: *sum of products (SOP)* and *product of sums (POS)*.

Sum of Products Form

The *sum of products (SOP)* is a Boolean expression containing AND terms, called *product terms*, of one or more literals each. The *sum* denotes the ORing of these terms. An example of a function expressed in sum of products form is:

$$F_1(x, y, z) = y' + xy + x'yz'$$

The expression has three product terms of one, two, and three literals each, respectively. Their sum is in effect an OR operation.

Conversion of Sum of Products into Canonical Sum of Products

It is sometimes needed to convert the Boolean function expressed as sum of products form into its canonical sum of products or sum of minterms form. It can be done by first inspecting each AND term to see if it contains all the variables. If it misses one or more variables, it is ANDed with an expression such as $x + x'$, where x is one of the missing variables. The following example clarifies the procedure:

$$F(A, B, C) = A + B'C$$

$$\begin{aligned}
 &= A(B + B')(C + C') + (A + A')B'C \\
 &= (AB + AB')(C + C') + AB'C + A'B'C \\
 &= ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C \\
 &= A'B'C + AB'C' + (AB'C + AB'C) + ABC' + ABC \\
 &= A'B'C + AB'C' + AB'C + ABC' + ABC \\
 &= m_1 + m_4 + m_5 + m_6 + m_7 = \sum(1,4,5,6,7)
 \end{aligned}$$

Product of Sums Form

A *product of sums (POS)* is a Boolean expression containing OR terms, called *sum terms*. Each term may have any number of literals. The *product* denotes the ANDing of these terms. An example of a function expressed in product of sums is:

$$F_2(x, y, z) = x(y' + z)(x' + y + z')$$

This expression has three terms of one, two, and three literals each. The product is an AND operation.

Conversion of Product of Sums into Canonical Product of Sums

To convert the Boolean function expressed as product of sums form into a canonical product of sums or product of maxterms form, an OR term having any missing variable x in it is ORed with xx' . This procedure is clarified by the following example:

$$\begin{aligned}
 F(x, y, z) &= (x' + y)(x + z)(y + z) \\
 &= (x' + y + zz')(x + yy' + z)(xx' + y + z) \\
 &= (x' + y + z)(x' + y + z')(x + y + z)(x + y' + z)(x + y + z)(x' + y + z) \\
 &= (x + y + z)(x + y + z)(x + y' + z)(x' + y + z)(x' + y + z)(x' + y + z') \\
 &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \\
 &= M_0 \cdot M_2 \cdot M_4 \cdot M_5 = \prod(0,2,4,5)
 \end{aligned}$$

3.5.4 Nonstandard Form

The canonical and noncanonical forms of Boolean functions discussed earlier are the standard forms of Boolean functions. However, a Boolean function may be expressed in a nonstandard form. For example, the function:

$$F_3(A, B, C, D) = (AB + CD)(A'B' + C'D')$$

is neither in sum of products nor in product of sums.

Conversion of Nonstandard Form into Standard Form

A Boolean function expressed in nonstandard form can be changed to a standard form by using the distributive law $x(y + z) = xy + xz$ or $x + yz = (x + y)(x + z)$. This procedure is clarified by the following example:

$$\begin{aligned}
 F_3(A, B, C, D) &= (AB + CD)(A'B' + C'D') \\
 &= (AB + CD)A'B' + (AB + CD)C'D' \\
 &= ABA'B' + CDA'B' + ABC'D' + CDC'D' \\
 &= A'B'CD + ABC'D'
 \end{aligned}$$

Again,

$$\begin{aligned}
 F_3(A, B, C, D) &= (AB + CD)(A'B' + C'D') \\
 &= (AB + C)(AB + D)(A'B' + C')(A'B' + D') \\
 &= (A + C)(B + C)(A + D)(B + D)(A' + C')(B' + C')(A' + D')(B' + D')
 \end{aligned}$$

3.6 Incompletely Specified Boolean Functions

In Boolean functions so far discussed, the value for the function is specified for all 2^n possible input combinations of n variables. These Boolean functions are said to be *completely specified* Boolean functions. Sometimes, however, for some input combinations the output may not be specified. It means that the outputs for these particular combinations are immaterial, as far as the operation of the corresponding digital circuit is concerned. This is also possible when certain input combinations can never occur due to some constraints. For example, in BCD code the four-bit combinations from 0000 to 1001 are assigned to decimal digits 0 to 9, and the combinations from 1010 to 1111 are unused. The outputs for these combinations are immaterial for the operation of the BCD code converter, since these combinations will never occur. Therefore, we don't care about the state of the output for these input combinations. Such situations are indicated on the truth table by writing x (or sometimes d) as the functional value, rather than 0 or 1. Such conditions are referred to as *don't care conditions* and the Boolean functions which include don't care conditions are said to be *incompletely specified* Boolean functions. For example, consider the Boolean function described by the truth table in Table 3.8. The function is an incompletely specified function and can be written as

$$f(x, y, z) = \sum (0, 4, 7) + \sum_{d.c.} (2, 6)$$

or

$$f(x, y, z) = \prod (1, 3, 5) \cdot \prod_{d.c.} (2, 6)$$

Table 3.8 Truth table of an incompletely specified Boolean function.

$x \ y \ z$	f
0 0 0	1
0 0 1	0
0 1 0	x
0 1 1	0
1 0 0	1
1 0 1	0
1 1 0	x
1 1 1	1

Exercises

3.1 Demonstrate by means of truth tables the validity of the following theorems of Boolean algebra.

(a) The associative laws.

(b) DeMorgan's theorems for three variables.

3.2 Using DeMorgan's theorems, convert the following expressions to one that has only single variable inversion.

(a) $F = ((A + B)C')'$ (b) $F = (RS'T + Q')'$

3.3 Obtain the truth table of the functions.

(a) $F = XY + XY' + Y'Z$ (b) $F = (X + Y')(X' + Y)(Y' + Z)$

3.4 Find the complement of the following Boolean functions.

(a) $F = (BC' + A'D)(AB' + CD')$

(b) $F = B'D + A'BC' + ACD + A'BC$

(c) $F = ((AB)'A)((AB)'B)$

(d) $F = AB' + C'D'$

3.5 Implement the following functions with logic gates:

(a) $F = xy + xy' + y'z$

(b) $F = (x + y')(x' + y)(y' + z)$

(c) $F = y(wz' + wz) + xy$

(d) $F = ((yz)' + w)' + w + yz + wx$

(e) $F = A'BC(A + D')$

(f) $F = AC + BC' + A'BC$

3.6 Expand the following function to their canonical forms.

(a) $F(A, B, C, D) = BC + AC' + AB + BCD$

(b) $F(A, B, C, D) = (A + C + D)(A + C + D')$

$(A + C' + D)(A + B')$

3.7 Express the following functions in a canonical sum of products and a canonical product of sums form.

(a) $F(A, B, C, D) = D(A' + B) + B'D$

(b) $F(w, x, y, z) = y'z + wxy' + wxz' + w'x'z$

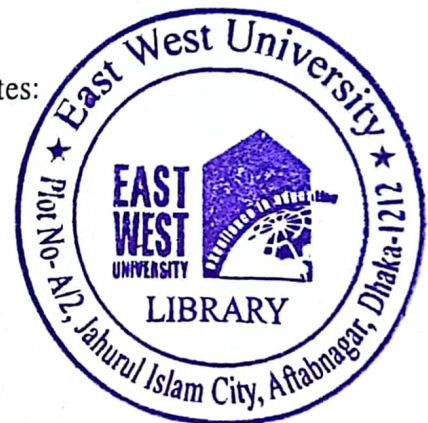
(c) $F(A, B, C, D) = (A + B' + C)(A + B')(A + C' + D')$
 $(A' + B + C + D')(B + C' + D')$

(d) $F(A, B, C) = (A' + B)(B' + C)$

(e) $F(x, y, z) = 1$

(f) $F(x, y, z) = (xy + z)(y + xz)$

3.8 Convert the following to the other canonical form.



- (a) $F(x, y, z) = \sum(1, 3, 7)$
 (b) $F(A, B, C, D) = \sum(0, 2, 6, 11, 13, 14)$
 (c) $F(x, y, z) = \prod(0, 3, 6, 7)$
 (d) $F(A, B, C, D) = \prod(0, 1, 2, 3, 4, 6, 12)$
- 3.9 What is the difference between canonical form and noncanonical form? Which form is preferable when implementing a Boolean function with gates? Which form is obtained when reading a function from a truth table?
- 3.10 The sum of all minterms of a Boolean function of n variables is 1. Prove the statement for $n = 3$.
- 3.11 The product of all maxterms of a Boolean function of n variables is 0. Prove the statement for $n = 3$.
- 3.12 Write the canonical sum of products and canonical product of sums function using short notation for the function described in the following truth table.

$A B C D$	F
0 0 0 0	0
0 0 0 1	0
0 0 1 0	1
0 0 1 1	1
0 1 0 0	0
0 1 0 1	x
0 1 1 0	0
0 1 1 1	1
1 0 0 0	x
1 0 0 1	x
1 0 1 0	1
1 0 1 1	1
1 1 0 0	0
1 1 0 1	0
1 1 1 0	1
1 1 1 1	0

Further Reading

1. E. J. McCluskey, *Logic Design Principles*, Prentice-Hall, Inc., 1986.
2. Zvi Kohavi, *Switching and Finite Automata Theory*, 2nd ed., McGraw-Hill Higher Education, 1990.
3. Richard S. Sandige, *Modern Digital Design*, McGraw-Hill Publishing Company, 1990.

4. M. Morris Mano, *Digital Logic and Computer Design*, Prentice-Hall, Inc., 2000.
5. John P. Uyemura, *Digital Systems Design: An Integrated Approach*, Brooks/Cole Publishing Company, 2000.
6. Ronald J. Tocci and Neal S. Widmer, *Digital Systems: Principles and Applications*, 8th ed., Prentice-Hall, Inc., 2001.
7. V. K. Jain, *Switching Theory and Digital Electronics*, Khanna Publishers, India, 2002.
8. Samuel C. Lee, *Digital Circuits and Logic Design*, Prentice-Hall, Inc., 2002.

CHAPTER 4 LOGIC GATES

4.1 Introduction

Logic gates are the basic building blocks of digital systems. *Logic gates* are electronic circuits that perform the operations of Boolean algebra. In this chapter, we shall describe logic gates and their operations using Boolean algebra. Integrated circuit (IC) logic families will also be introduced.

4.2 Logic Operations

When the binary operators AND and OR are placed between two variables x and y , they form two Boolean functions $x \cdot y$ and $x + y$, respectively. It was stated in the previous chapter that there are 2^{2^n} functions for n binary variables. For two variables, $n = 2$ and the number of possible Boolean functions is 16. Therefore, the AND and OR functions are only two of a total of 16 possible functions formed with two binary variables.

The truth tables for the 16 functions formed with two binary variables x and y are listed in Table 4.1. In this table, each of the 16 columns F_0 to F_{15} represents a truth table of one possible function for two given variables x and y . Note that the functions are determined from the 16 binary combinations that can be assigned to F .

Table 4.1 Truth tables for the 16 functions of two binary variables.

xy	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
00	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
01	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
10	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
11	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

The 16 functions listed in truth table form can be expressed algebraically by means of Boolean expression. This is shown in the first column of Table 4.2. The Boolean expressions listed are in their simplified form. Although each function can be expressed in terms of the Boolean operators AND, OR, and NOT, there is no reason one cannot assign special operator symbols for expressing the other functions. Such operator symbols are listed in the second column of Table 4.2. However, all the new symbols shown, except for the EXOR symbol $\oplus$, are not in common use by digital designers.

Table 4.2 Boolean expressions for the 16 functions of two variables.

Boolean functions	Operator symbol	Name	Comment
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x / y	Inhibition	x but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y / x	Inhibition	y but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	EXOR	x or y but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$x \otimes y$	EXNOR (Equivalence)	x equals y
$F_{10} = y'$	y'	NOT (Inversion)	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y then x
$F_{12} = x'$	x'	NOT (Inversion)	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

Each of the functions in Table 4.2 is listed with an accompanying name and a comment that explains the function in some way. The 16 functions listed can be subdivided into three categories:

1. Two functions (F_0 and F_{15}) that produce a constant 0 or 1.
2. Four functions (F_3 , F_5 , F_{10} , and F_{12}) with unary operators NOT and transfer.
3. Ten functions (F_1 , F_2 , F_4 , F_6 , F_7 , F_8 , F_9 , F_{11} , F_{13} , and F_{14}) with binary operators that define eight different operations AND (F_1), OR (F_7), NAND (F_{14}), NOR (F_8), EXOR (F_6), EXNOR (F_9), inhibition (F_2 and F_4), and implication (F_{11} and F_{13}).

Any binary function can be equal to 0 or 1. The NOT function produces the complement of each of the binary variables. A function that is equal to an input

variable has been given the name *transfer*, because the variable x or y is transformed through the gate that forms the function without changing its value. Of the eight binary operators, two (inhibition and implication) are used by logicians but are seldom used in digital logic. The AND and OR operators have been mentioned in conjunction with Boolean algebra. The other four operators such as NAND, NOR, EXOR, and EXNOR are extensively used in the design of digital systems.

The NOR function is the complement of the OR function and its name is an abbreviation of *not-OR*. Similarly, NAND is the complement of AND and is an abbreviation of *not-AND*. The EXOR is an abbreviation of *exclusive-OR* and is similar to OR but excludes the combination of *both* x and y being equal to 1. The EXNOR or *equivalence* is a function that is 1 when the two binary variables are equal, i.e., when both are 0 or both are 1. EXNOR is an abbreviation of *exclusive-OR-not*. The EXOR and EXNOR are the complements of each other. This can be easily verified by inspecting Table 4.1. The truth table for the EXOR is F_6 and for the EXNOR is F_9 , and these two functions are the complements of each other.

4.3 Logic Gates

Since Boolean functions are expressed in terms of AND, OR, and NOT operations, it is easier to implement a Boolean function with these types of gates. The possibility of constructing gates for the other logic operations is of practical interest. Factors to be weighed when considering the construction of other types of logic gates are

- (1) the feasibility and economy of producing the gate with physical electronic components,
- (2) the possibility of extending the gate to more than two inputs,
- (3) the basic properties of the binary operator such as commutativity and associativity, and
- (4) the ability of the gate to implement Boolean functions alone or in conjunction with other gates.

Of the 16 functions defined in Table 4.2 two are equal to a constant and four others are repeated twice. There are only ten functions left to be considered as candidates for logic gates. Two, inhibition and implication, are not commutative or associative and thus are impractical to use as standard logic gates. The other eight: transfer, NOT, AND, OR, NAND, NOR, EXOR, and EXNOR, are used as standard gates in digital design. The graphic symbols and truth tables of the eight gates are shown in Figure 4.1. Each gate has one or two binary input variables designated by x and y and one binary output variable designated by F .

A buffer produces the *transfer* function but does not produce any particular logic operations, since the binary value of the output is equal to the binary value of the input. This is used merely for power amplification of the signal. The NOT gate inverts the logic sense of a binary variable. It produces the complement function. The triangle symbol designates a buffer and the small circle in the output of the graphic buffer symbol designates the logic complement.

The AND and OR gates were defined in Figure 1.4. The NAND function is the complement of the AND function, as indicated by a graphic symbol which consists

of an AND graphic symbol followed by a small circle. The NOR function is the complement of the OR function and uses an OR graphic symbol followed by a small circle. The NAND and NOR gates are extensively used as standard logic gates and are in fact far more popular than the AND and OR gates. This is because NAND and NOR gates are easily constructed with transistor circuits and because these gates are *universal* gates, that is, Boolean functions can be easily implemented with them.

The EXOR gate has a graphic symbol similar to that of the OR gate, except for the additional curved line on the input side. The EXNOR gate is the complement of the EXOR gate, as indicated by the small circle on the output side of the graphic symbol. The EXOR gates are finding more and more use in digital logic design.

Name	Graphic symbol	Algebraic function	Truth table															
Buffer (transfer)	$x \rightarrow F$	$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
NOT (inverter)	$x \rightarrow F$	$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
AND	$x, y \rightarrow F$	$F = xy$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR	$x, y \rightarrow F$	$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NAND	$x, y \rightarrow F$	$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR	$x, y \rightarrow F$	$F = (x + y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
EXOR	$x, y \rightarrow F$	$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
EXNOR (equivalence)	$x, y \rightarrow F$	$F = xy + x'y'$ $= x \otimes y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Figure 4.1 Digital logic gates.

Extension to Multiple Inputs

The gates shown in Figure 4.1, except for the buffer and NOT, can be extended to have more than two inputs. A gate can be extended to have multiple inputs if the binary operation it represents is commutative and associative. The AND and OR operations, defined in Boolean algebra, possess these two properties. For the AND and OR functions we have:

$$\begin{aligned}x \cdot y &= y \cdot x \\x + y &= y + x\end{aligned}\quad \text{commutative}$$

and

$$\begin{aligned}(x \cdot y) \cdot z &= x \cdot (y \cdot z) = x \cdot y \cdot z \\(x + y) + z &= x + (y + z) = x + y + z\end{aligned}\quad \text{associative}$$

which indicates that the gate inputs can be interchanged and that the AND and OR functions can be extended to three or more variables. Multiple-input AND and OR gates were introduced in Figure 1.4.

The construction of three-input AND and OR gates by cascading two-input gates is shown in Figure 4.2(a) and (c) respectively. Graphically, they can be represented with a simple three-input gate as shown in Figure 4.2(b) and (d) respectively.

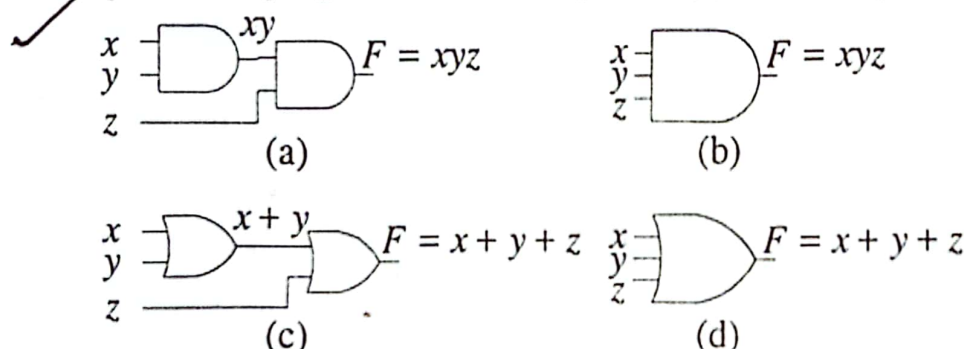


Figure 4.2 Three-input AND and OR gates – (a) 3-input AND gate using 2-input AND gates, (b) graphic symbol of 3-input AND gate, (c) 3-input OR gate using 2-input OR gates, and (d) graphic symbol of 3-input OR gate.

The NAND and NOR functions are commutative and their gates can be extended to have more than two inputs, provided the definitions of the operation is slightly modified. The difficulty is that the NAND and NOR operations are not associative, i.e., $(x \uparrow y) \uparrow z \neq x \uparrow (y \uparrow z)$ and $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$, as shown below:

$$\begin{aligned}(x \uparrow y) \uparrow z &= ((xy)' \cdot z)' = (xy) + z' = (x + z')(y + z') \\x \uparrow (y \uparrow z) &= (x \cdot (yz)')' = x' + (yz) = (x' + y)(x' + z)\end{aligned}$$

and

$$(x \downarrow y) \downarrow z = ((x + y)' + z)' = (x + y) \cdot z' = xz' + yz'$$

$$x \downarrow (y \downarrow z) = (x + (y + z)')' = x' \cdot (y + z) = x'y + x'z$$

To overcome this difficulty, we define the multiple-input NAND and NOR gates as complemented AND and OR gates respectively. Thus, by definition, we have:

$$x \uparrow y \uparrow z = (xyz)'$$

$$x \downarrow y \downarrow z = (x + y + z)'$$

The graphic symbols for the three-input NAND and NOR gates are shown in Figure 4.3.

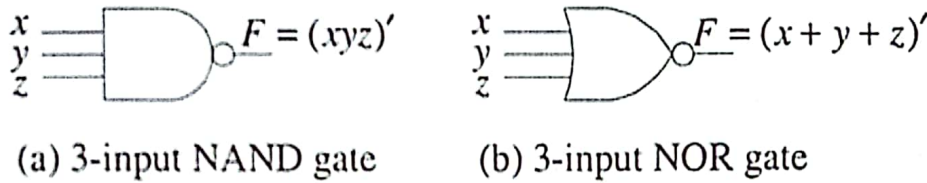


Figure 4.3 Multiple-input NAND and NOR gates.

As the NAND and NOR operations are not associative, in writing cascaded NAND and NOR operations, one must use the correct parentheses to signify the proper sequence of the gates. To demonstrate this, consider the circuit of Figure 4.4. The Boolean function for the circuit of Figure 4.4(a) must be written as:

$$F = ((ABC)'(DE)')' = ABC + DE$$

and the Boolean function for the circuit of Figure 4.4(b) must be written as:

$$F = ((A + B + C)' + (D + E)')' = (A + B + C)(D + E)$$

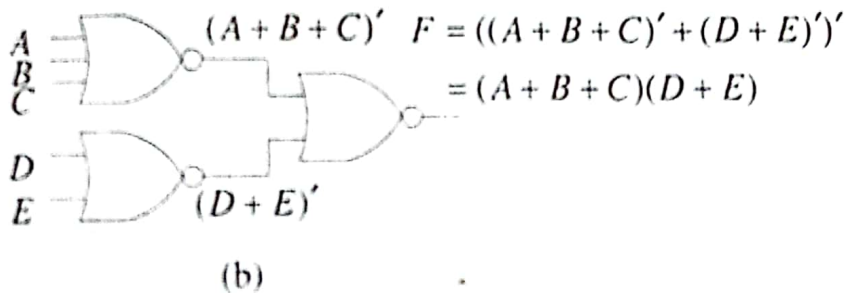
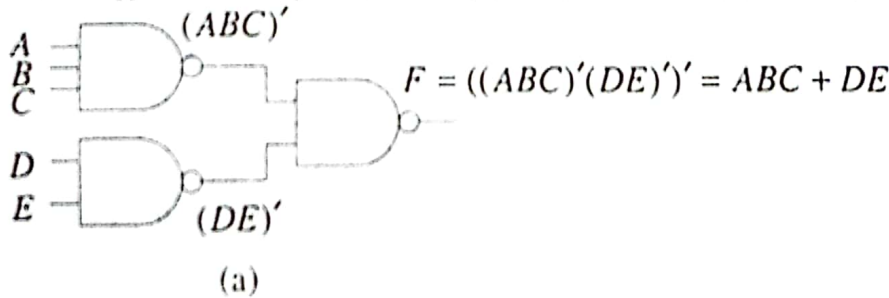


Figure 4.4 (a) Cascaded NAND gates and (b) cascaded NOR gates.

The EXOR and EXNOR functions are both commutative and associative, that is,

$$A \oplus B = B \oplus A$$

$$A \otimes B = B \otimes A$$

commutative

and

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

$$(A \otimes B) \otimes C = A \otimes (B \otimes C) = A \otimes B \otimes C$$

associative

Because of these two properties, we can extend EXOR and EXNOR gates to more than two inputs. However, multiple-input EXOR and EXNOR gates are uncommon from the hardware standpoint. In fact, even a two-input gate is usually constructed with other types of gates. Moreover, the definition of these functions must be modified when extended to more than two variables. The EXOR is an *odd* function, i.e., it is equal to 1 if the input variables have an odd number of 1s. An n -variable EXOR expression is equal to the Boolean function with $2^n / 2$ minterms whose equivalent binary numbers have an odd number of 1s. For example,

$$\begin{aligned} A \oplus B \oplus C \oplus D &= (AB' + A'B) \oplus (CD' + C'D) \\ &= (AB' + A'B)(CD' + C'D)' + (AB' + A'B)'(CD' + C'D) \\ &= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D) \\ &= AB'CD + AB'C'D' + A'BCD + A'BC'D' + ABCD' \\ &\quad + ABC'D + A'B'CD' + A'B'C'D \\ &= \sum(1, 2, 4, 7, 8, 11, 13, 14) \end{aligned}$$

The EXNOR is an *even* function, i.e., it is equal to 1 if the input variables have an even number of 0s. An n -variable EXNOR expression is equal to the Boolean function with $2^n / 2$ minterms, whose equivalent binary numbers have an even number of 0s. For example,

$$\begin{aligned} A \otimes B \otimes C \otimes D &= (AB + A'B') \otimes (CD + C'D') \\ &= (AB + A'B')(CD + C'D')' + (AB + A'B')'(CD + C'D') \\ &= (AB + A'B')(CD + C'D') + (AB' + A'B)(CD' + C'D) \\ &= ABCD + ABC'D' + A'B'CD + A'B'C'D' + AB'CD' \\ &\quad + AB'C'D + A'BC'D + AB'C'D \\ &= \sum(0, 3, 5, 6, 9, 10, 12, 15) \end{aligned}$$

When the number of variables in a function is even, EXOR expression and EXNOR expression are complement of each other. This is evident from the above discussion, because $A \oplus B \oplus C \oplus D = \sum(1, 2, 4, 7, 8, 11, 13, 14)$ and $A \otimes B \otimes C \otimes D = \sum(0, 3, 5, 6, 9, 10, 12, 15)$. When the number of variables in a

function is odd, the minterms with an even number of 0s are the same as the minterms with an odd number of 1s. Therefore, in this case, EXOR expression is equal to EXNOR expression. For example,

$$\begin{aligned}
 A \oplus B \oplus C &= A \oplus (BC' + B'C) \\
 &= A(BC' + B'C)' + A'(BC' + B'C) \\
 &= A(BC + B'C') + A'(BC' + B'C) \\
 &= ABC + AB'C' + A'BC' + A'B'C \\
 &= \sum(1,2,4,7)
 \end{aligned}$$

and

$$\begin{aligned}
 A \otimes B \otimes C &= A \otimes (BC + B'C') \\
 &= A(BC + B'C') + A'(BC + B'C')' \\
 &= A(BC + B'C') + A'(BC' + B'C) \\
 &= ABC + AB'C' + A'BC' + A'B'C \\
 &= \sum(1,2,4,7)
 \end{aligned}$$

The construction of three-input EXOR and EXNOR gates is shown in Figure 4.5. They are normally implemented by cascading two-input gates as shown in Figure 4.5(a) and (c). Graphically, they can be represented with a single three-input gate as shown in Figure 4.5(b) and (d). The truth tables in Figure 4.5(e) clearly indicate the output of the gates.

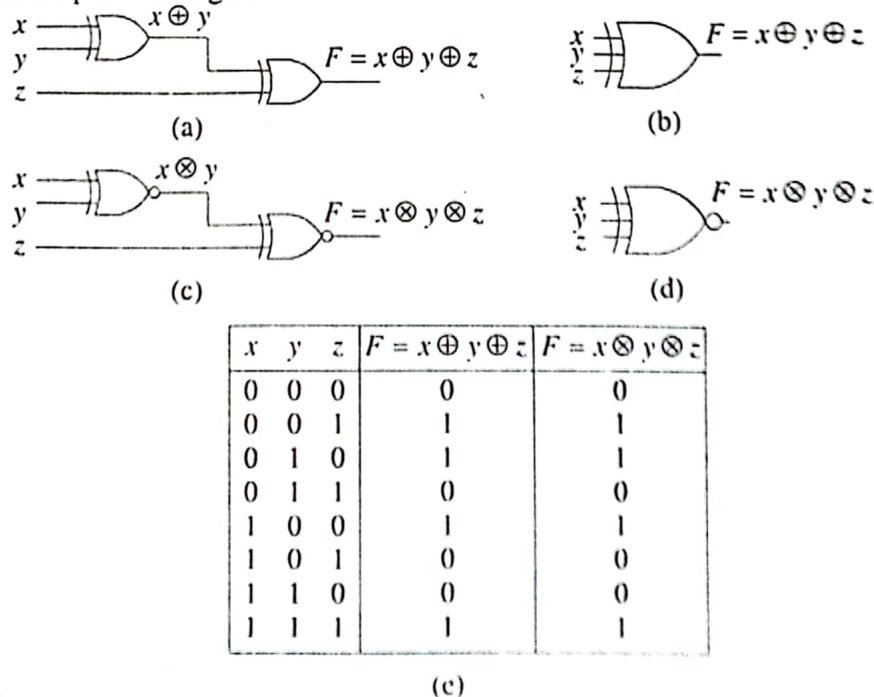


Figure 4.5 Three-input EXOR and EXNOR gates – (a) 3-input EXOR gate using 2-input EXOR gates, (b) graphic symbol of 3-input EXOR gate, (c) 3-input EXNOR gate using 2-input EXNOR gates, (d) graphic symbol of 3-input EXNOR gate, and (e) truth table of 3-input EXOR and EXNOR gates.

4.4 Universality of NAND and NOR Gates

All Boolean expressions consist of various combinations of the basic operations of AND, OR, and NOT. Therefore, any expression can be implemented using combinations of AND gates, OR gates, and NOT gates. It is possible, however, to implement any logic expression using only NAND gates and no other type of gate. This is because NAND gates, in the proper combination, can be used to perform each of the Boolean operations AND, OR, and NOT. This is demonstrated in Figure 4.6. In Figure 4.6(a), we have a two-input NAND gate whose inputs are purposely connected together so that the variable A is applied to both. In this configuration, the NAND simply acts as NOT, since its output is $F = (A \cdot A)' = A'$. In Figure 4.6(b), we have two NAND gates connected so that the AND operation is performed. NAND gate 2 is used as a NOT gate to change $(AB)'$ to $((AB)')' = AB$, which is the desired AND function. In Figure 4.6(c), we have three NAND gates connected so that the OR operation is performed. Here NAND gates 1 and 2 are used as NOT gates to invert the inputs, so that the final output is $F = (A' \cdot B')' = A + B$, which is the desired OR function.

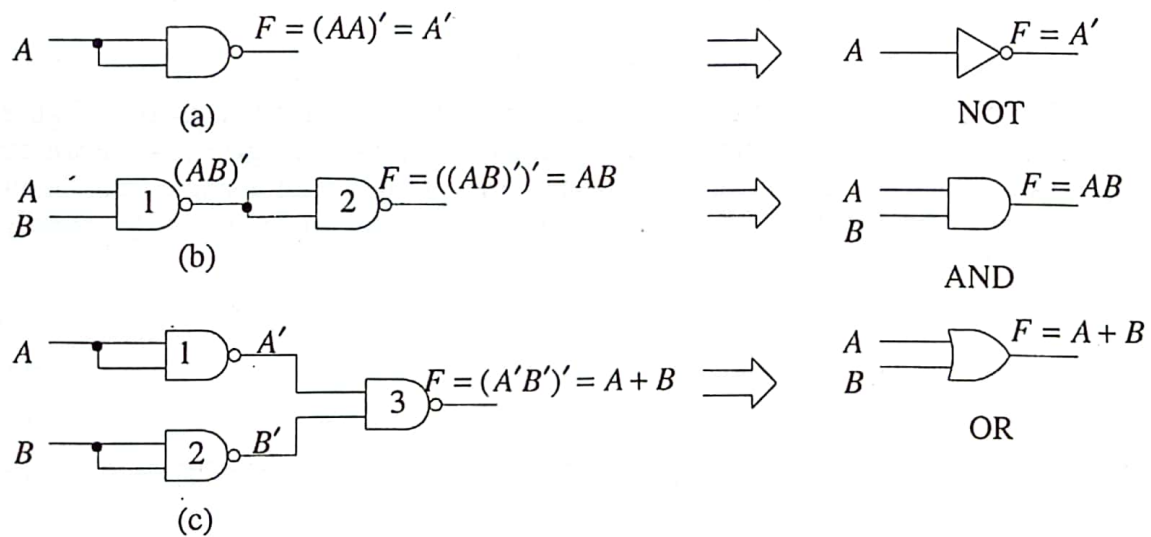


Figure 4.6 NAND gates can be used to implement (a) NOT, (b) AND, and (c) OR gates.

In a similar manner, it can be shown that NOR gates can be arranged to implement AND, OR, and NOT operations. This is illustrated in Figure 4.7. In Figure 4.7(a), NOR gate with its inputs connected together behaves as a NOT gate, since the output is $F = (A + A)' = A'$. In Figure 4.7(b), two NOR gates are arranged so that the OR operation is performed. NOR gate 2 is used as a NOT gate to change $(A + B)'$ to $((A + B)')' = A + B$, which is the desired OR function. In Figure 4.7(c), three NOR gates are used to implement AND operation. Here NOR gates 1 and 2 are used as NOT gates to invert the inputs, so that the final output is $F = (A' + B')' = AB$, which is the desired AND function.

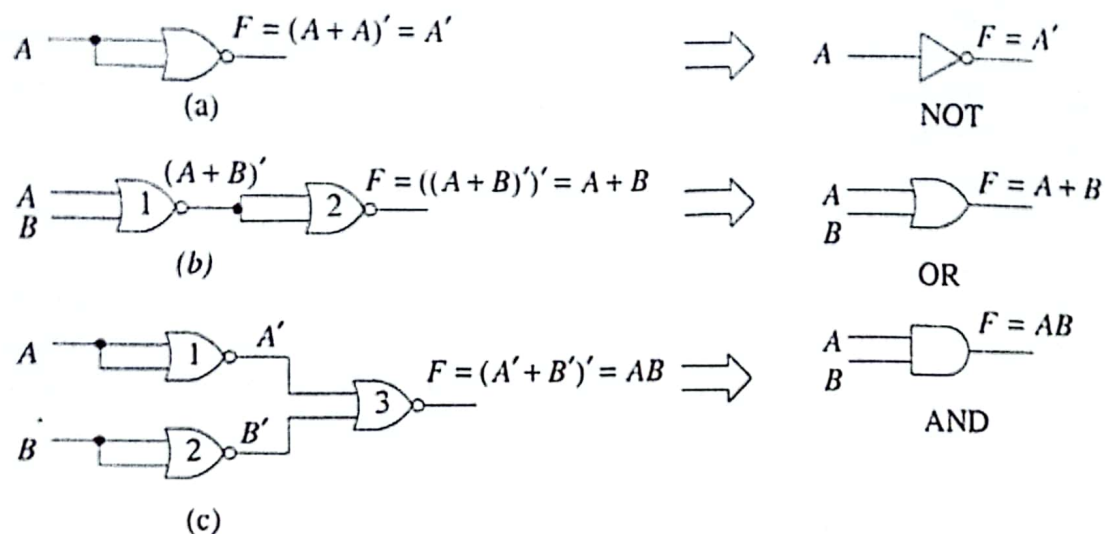
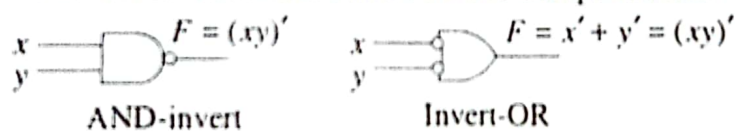


Figure 4.7 NOR gates can be used to implement (a) NOT, (b) OR, and (c) AND gates.

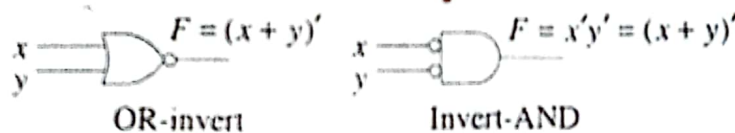
Since any of the Boolean operations can be implemented using only NAND gates, any logic circuit can be constructed using only NAND gates. The same is true for NOR gates. We shall discuss these topics in detail in Chapter 6.

4.5 Alternative Graphic Symbols for NAND, NOR, and NOT Gates

It is convenient in many cases to define two other graphic symbols for NAND and NOR gates. Two equivalent symbols for the NAND gate are shown in Figure 4.8(a). The AND-invert symbol has been defined previously and consists of an AND graphic symbol followed by a small circle. Instead, it is possible to represent a NAND gate by an OR graphic symbol preceded by small circles in all the inputs. The invert-OR symbol for the NAND gate follows from DeMorgan's theorem and from the convention that small circles denote complementation.



(a) Two graphic symbols for NAND gate



(b) Two graphic symbols for NOR gate



(c) Three graphic symbols for NOT gate



(d) Three alternate graphic symbols for NOT gate

Figure 4.8 Alternative graphic symbols for NAND, NOR, and NOT gates.

Similarly, there are two graphic symbols for the NOR gate as shown in Figure 4.8(b). The OR-invert is the conventional symbol. The invert-AND is a convenient alternative that utilizes DeMorgan's theorem and the convention that small circle in the inputs denote complementation.

A two-input NAND or NOR gate with its input connected together behaves like NOT gate. As a consequence, a NOT gate can be drawn in three different ways as shown in Figure 4.8(c). The small circles in all NOT symbols can be transferred to the input terminal without changing the logic of the gate as shown in Figure 4.8(d).

These alternative graphic symbols for NAND and NOR gates are very useful in NAND and NOR circuit implementation, which will be discussed in detail in Chapter 6.

4.6 EXOR and EXNOR Gates

4.6.1 Properties of EXOR and EXNOR Operations

The EXOR and EXNOR operations are finding more and more applications in digital systems design. We shall discuss EXOR circuits in detail in Chapter 6. However, some basic properties of EXOR and EXNOR operations are stated in the following theorems.

Theorems

- | | |
|------------------------------------|-----------------------------------|
| 1. (a) $X \oplus X = 0$ | (b) $X \otimes X = 1$ |
| 2. (a) $X \oplus X' = 1$ | (b) $X \otimes X' = 0$ |
| 3. (a) $1 \oplus X = X'$ | (b) $0 \otimes X = X'$ |
| 4. (a) $X \oplus Y = X' \oplus Y'$ | (b) $X \otimes Y = X' \otimes Y'$ |

The proofs of the above theorems are given below.

- (a) $X \oplus X = XX' + X'X = 0$
 - (b) $X \otimes X = XX + X'X' = X + X' = 1$
- (a) $X \oplus X' = X(X')' + X'X' = X + X' = 1$
 - (b) $X \otimes X' = XX' + X'(X')' = XX' + X'X = 0$
- (a) $1 \oplus X = 1 \cdot X' + 1' \cdot X = 1 \cdot X' + 0 \cdot X = X'$
 - (b) $0 \otimes X = 0 \cdot X + 0' \cdot X' = 0 \cdot X + 1 \cdot X' = X'$
- (a) From 1(a), we have that $X' \otimes X = 0$. To prove $X \oplus Y = X' \oplus Y'$, we shall prove $(X \oplus Y) \oplus (X' \oplus Y') = 0$.

$$\begin{aligned}
 (X \oplus Y) \oplus (X' \oplus Y') &= X \oplus Y \oplus X' \oplus Y' && \text{using associative law} \\
 &= X \oplus X' \oplus Y \oplus Y' && \text{using commutative law} \\
 &= 1 \oplus 1 && \text{using theorem 2(a)} \\
 &= 0 && \text{using theorem 1(a)}
 \end{aligned}$$

(b) From 1(b), we know that $X \otimes X = 1$. To prove $X \otimes Y = X' \otimes Y'$, we shall prove $(X \otimes Y) \otimes (X' \otimes Y') = 1$.

$$\begin{aligned}
 (X \otimes Y) \otimes (X' \otimes Y') &= X \otimes Y \otimes X' \otimes Y' \quad \text{using associative law} \\
 &= X \otimes X' \otimes Y \otimes Y' \quad \text{using commutative law} \\
 &= 0 \otimes 0 \quad \text{using theorem 2(b)} \\
 &= 1 \quad \text{using theorem 1(b)}
 \end{aligned}$$

4.6.2 EXOR and EXNOR Gate Implementations

As discussed in section 4.3, a two-input EXOR gate is usually constructed with other types of gates. Implementation of a two-input EXOR gate is shown in Figure 4.9. In Figure 4.9(a), the output of AND gate 1 is xy' and the output of AND gate 2 is $x'y$. The final output of OR gate is $xy' + x'y$, which is the required EXOR expression. In Figure 4.9(b), the output of NAND gate 1 is $(xy)'$. The output of NAND gate 2 is $(x(xy)')' = (x(x' + y'))' = (xy')'$. The output of NAND gate 3 is $(y(xy)')' = (y(x' + y'))' = (x'y)'$. The final output of NAND gate 4 is $((xy')'(x'y)')' = xy' + x'y$, which is the required EXOR expression. In Figure 4.9(c), NOR gate 1 and 2 act as inverter. The output of NOR gate 3 is $(x + y)'$ and the output of NOR gate 4 is $(x' + y')'$. The final output of NOR gate 5 is $((x + y)' + (x' + y')')' = (x + y)(x' + y') = xy' + x'y$, which is the required EXOR expression.

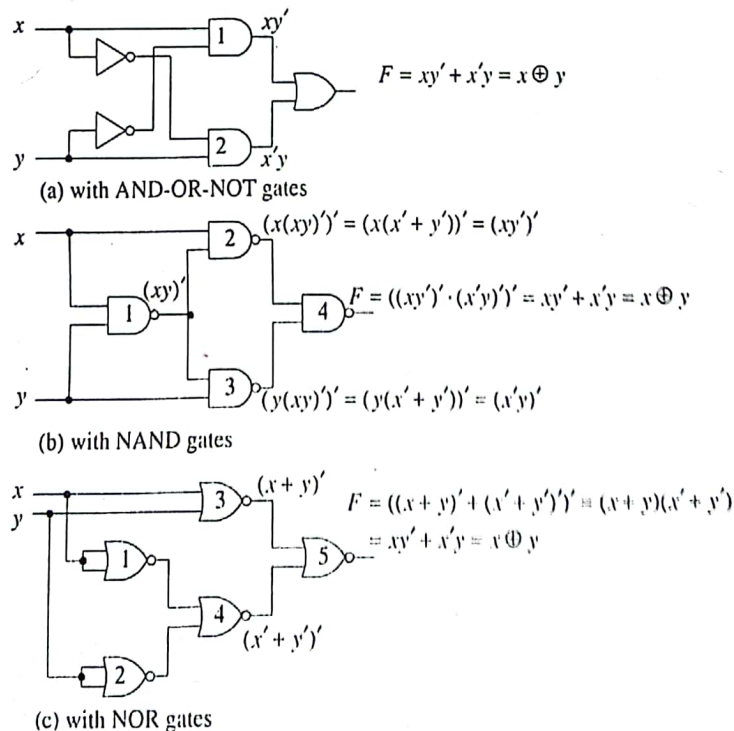


Figure 4.9 Implementation of EXOR gate with other gates.

Implementation of a two-input EXNOR gate is shown in Figure 4.10. In Figure 4.10(a), the output of AND gate 1 is xy and the output of AND gate 2 is $x'y'$. The final output of OR gate is $xy + x'y'$, which is the required EXNOR expression. In Figure 4.10(b), NAND gate 1 and 2 act as inverter. The output of NAND gate 3 is $(xy)'$ and the output of NAND gate 4 is $(x'y')'$. The final output of NAND gate 5 is $((xy)' \cdot (x'y')')' = xy + x'y'$, which is the required EXNOR expression. In Figure 4.8(c), the output of NOR gate 1 is $(x + y)'$. The output of NOR gate 2 is $(x + (x + y)')' = x'(x + y) = x'y$. The output of NOR gate 3 is $((x + y)' + y)' = (x + y)y' = xy'$. The final output of NOR gate 4 is $(x'y + xy')' = xy + x'y'$, which is the required EXNOR expression.

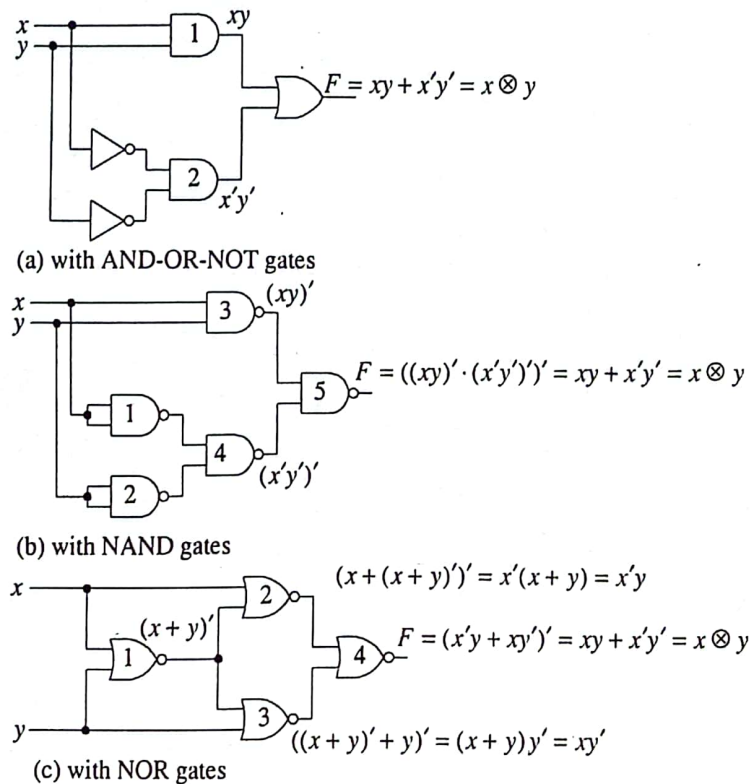


Figure 4.10 Implementation of EXNOR gate with other gates.

4.7 Integrated Circuit Logic Gates

4.7.1 IC Logic Families

Digital circuits are invariably constructed with *integrated circuits (ICs)*. Digital IC gates are classified by the specific logic-circuit family to which they belong. Each logic family has its own basic electronic circuit upon which more complex digital circuits and functions are developed. The basic circuit in each family is either a NAND or a NOR gate. The electronic components employed in the construction of the basic circuit are usually used to name the logic family. Many different logic families of digital ICs have been introduced commercially. The ones that have achieved widespread popularity are listed below:

TTL	Transistor-transistor logic
ECL	Emitter-coupled logic
MOS	Metal-oxide semiconductor
CMOS	Complementary metal-oxide semiconductor
I ² L	Integrated-injection logic

TTL has an extensive list of digital functions and is currently the most popular logic family. ECL is used in systems requiring high-speed operations. MOS and I²L are used in circuits requiring high component density. CMOS is used in systems requiring low power consumption.

Because of the high density with which transistors can be fabricated in MOS and I²L, these two families are mostly used for LSI functions. The other three families, TTL, ECL, and CMOS, have LSI devices and also a large number of MSI and SSI devices.

SSI devices are those that come with a small number of gates in one IC package. Some typical TTL SSI circuits are shown in Figure 4.11. Each IC is enclosed with a 14-pin package. The pins are numbered along the two sides of the package and specify the connections that can be made. The gates drawn inside the ICs are for information only and cannot be seen.

TTL ICs are usually distinguished by numerical designation as the 5400 and 7400 series. The former has a wide operating-temperature range, suitable for military use, and the latter has a narrower temperature range, suitable for industrial use. The numeric designation of the 7400 series means that IC package are numbered as 7400, 7401, 7402, etc. Some vendors make available TTL ICs with different numerical designation, such as the 9000 or the 8000 series. Figure 4.11 shows two TTL SSI circuits. The 7404 provides six (hex) NOT gates (inverters) in a package. The 7400 provides four (quadruple) 2-input NAND gates. The terminal marked V_{CC} and GND are the power supply pins which require a voltage of 5 volts for proper operation.

The most common ECL type is designated as the 10000 series and the CMOS circuits are designated as the 4000 series.

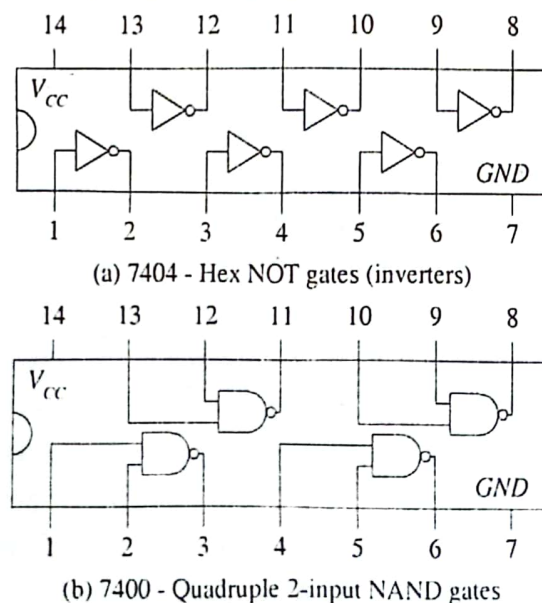


Figure 4.11 Some typical TTL SSI gates.

4.7.2 Logic Conventions

The binary signal at the inputs or output of any gate can have one of two values, except during transition. One signal value represents 1 and the other 0. Since two signal values are assigned to two logic values, there exist two different assignments of signals to logic. Because of the principle of duality of Boolean algebra, an interchange of signal-value assignment results in a dual-function implementation. Consider the two values of a binary signal as shown in Figure 4.12. One value must be higher than the other since the two values must be different in order to distinguish between them. We designate the higher level by H and the lower level by L . There are two choices for logic-value assignment. Choosing the high-level H to represent 1, as shown in Figure 4.12(a), defines a *positive-logic* system. Choosing the low-level L to represent 1, as shown in Figure 4.12(b), defines a *negative-logic* system.

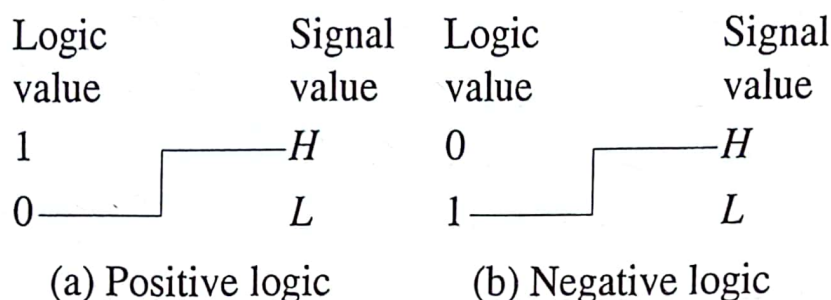


Figure 4.12 Signal-amplitude assignment and type of logic.

Integrated-circuit data sheets define digital functions not in terms of 1 or 0, but rather in terms of H and L levels. It is up to the user to decide on a positive or negative logic assignment. In TTL, supply voltage $V_{CC} = 5$ volts, high-level voltage range is 2.4 – 5 volts, and low-level voltage range is 0 – 0.4 volts. It has typical values of $H = 3.5$ volts and $L = 0.2$ volts.

In light of this discussion, it would be necessary to justify the logic symbols used for the ICs listed in Figure 4.11. Take, for example, one of the gates of the 7400 IC. This gate is shown in block diagram form in Figure 4.13(a). The manufacturer's truth table for this gate given in a data sheet is shown in Figure 4.13(b). This specifies the physical behavior of the gate, with H being typically 3.5 volts and L being 0.2 volt. This physical gate can function as either a NAND or NOR gate, depending on the polarity assignment.

The truth table of Figure 4.13(c) assumes positive-logic assignment with $H = 1$ and $L = 0$. Checking this truth table with the truth tables in Figure 4.1, we recognize it as a NAND gate. The graphic symbol for a positive-logic NAND gate is shown in Figure 4.13(d) and is similar to the one adopted previously.

Now consider the negative-logic assignment for this physical gate with $L = 1$ and $H = 0$. The result is the truth table shown in Figure 4.13(e). This table can be recognized to represent the NOR function even though its entries are listed backwards. The graphic symbol for a negative-logic NOR gate is shown in Figure 4.13(f). The small triangle in the input and output wires designates a *polarity indicator*. The presence of this polarity indicator along a terminal indicates that

negative logic is assigned to the terminal. Thus, the same physical gate can function either as a positive-logic NAND or as a negative-logic NOR. The one drawn in the diagram is completely dependant on the polarity assignment that the designer wishes to employ.

In a similar manner, it is possible to show that a positive-logic NOR is the same physical gate as a negative-logic NAND. The same relation holds between AND and OR gates or between EXOR and EXNOR gates. In any case, if negative logic is assumed in any input or output terminal, it is necessary to include the polarity indicator triangle symbol along the terminal. Note that the ICs presented in Figure 4.11 are shown with their positive-logic graphic symbols.

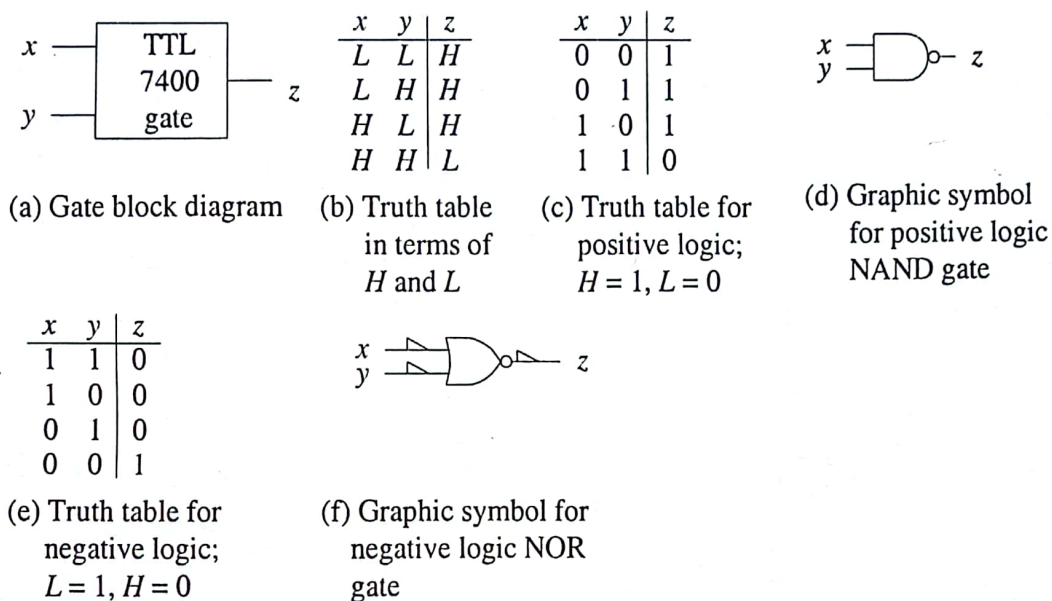


Figure 4.13 Demonstration of positive and negative logic.

4.7.3 Characteristics of IC Logic Families

The most important parameters of a IC logic family are *fan-out*, *power dissipation*, *propagation delay*, and *noise margin*.

The *fan-out* is the maximum number of inputs that can be connected to the output of a gate, and it is expressed by a number. Exceeding the fan-out capacity may cause a malfunction of the gate. The fan-out capacity of a gate must be considered when simplifying Boolean functions. Care must be taken not to develop expressions that result in an overloaded gate. Buffers are sometimes employed to provide additional driving capabilities for heavy loads.

Power dissipation represents the actual power dissipated in the gate and is expressed in milliwatts (mW). The total power dissipated in a system is the sum total of the power dissipated in all ICs.

The signals through a gate take a certain amount of time to propagate from the inputs to the output. This interval of time is defined as the *propagation delay* of the gate. Propagation delay is expressed in nanoseconds (ns). The signals that travel from the inputs of a digital circuit to its output pass through a series of gates. The

sum of the propagation delays through the gates is the total propagation delay of the circuit. When speed of operation is important, each gate must have a small propagation delay and the digital circuit must have a minimum number of series gates between inputs and outputs.

Noise is a term used to denote an undesirable signal that is superimposed upon the normal operating signal. *Noise margin* represents the maximum noise signal that can be tolerated by the gate and is expressed in volts (V).

There are many versions of TTL, the common are standard TTL, Schottky TTL, and low-power Schottky TTL. The typical characteristics of these versions are listed in Table 4.3.

Table 4.3 Typical characteristics of TTL logic family.

Version of TTL logic family	Fan out	Power dissipation (mW)	Propagation delay (nS)	Noise margin (V)
Standard TTL	10	10	10	0.4
Schottky TTL	10	22	3	0.4
Low-power Schottky TTL	20	2	10	0.4

4.8 Some Available TTL Logic Gates

Some of the available TTL logic gates are listed in Table 4.4. The names of the gates are given for positive-logic convention and their pin connection is shown in Figure 4.14. In Figure 4.14, 1, 2, etc. represent gate number within the IC; A, B, C etc. represent inputs of the corresponding gate; Y represents output of the corresponding gate; and NC represents "No Connection" pin, that is, the pin is not used. For example, 2A represents input A of gate 2 and 2Y represents output of gate 2.

Table 4.4 Some of the available TTL IC gates.

IC No	Description	IC No	Description
7400	Quad 2-input NAND gates	7421	Dual 4-input AND gates
7402	Quad 2-input NOR gates	7427	Triple 3-input NOR gates
7404	Hex NOT gates (inverters)	7430	8-input NAND gate
7408	Quad 2-input AND gates	7432	Quad 2-input OR gates
7410	Triple 3-input NAND gates	7486	Quad 2-input EXOR gates
7411	Triple 3-input AND gates	74133	13-input NAND gate
7420	Dual 4-input NAND gates	74266	Quad 2-input EXNOR gates

1	1A	V_{CC}	14
2	1B	4B	13
3	1Y	4A	12
4	2A	4Y	11
5	2B	3B	10
6	2Y	3A	9
7	GND	3Y	8

7400 (4 2-In NAND)

1	1Y	V_{CC}	14
2	1A	4Y	13
3	1B	4B	12
4	2Y	4A	11
5	2A	3Y	10
6	2B	3B	9
7	GND	3A	8

7402 (4 2-In NOR)

1	1A	V_{CC}	14
2	1Y	6A	13
3	2A	6Y	12
4	2Y	5A	11
5	3A	5Y	10
6	3Y	4A	9
7	GND	4Y	8

7404 (6 NOT)

1	1A	V_{CC}	14
2	1B	4B	13
3	1Y	4A	12
4	2A	4Y	11
5	2B	3B	10
6	2Y	3A	9
7	GND	3Y	8

7408 (4 2-In AND)

1	1A	V_{CC}	14
2	1B	1C	13
3	2A	1Y	12
4	2B	3C	11
5	2C	3B	10
6	2Y	3A	9
7	GND	3Y	8

7410 (3 3-In NAND)

1	1A	V_{CC}	14
2	1B	1C	13
3	2A	1Y	12
4	2B	3C	11
5	2C	3B	10
6	2Y	3A	9
7	GND	3Y	8

7411 (3 3-In AND)

1	1A	V_{CC}	14
2	1B	2D	13
3	NC	2C	12
4	1C	NC	11
5	1D	2B	10
6	1Y	2A	9
7	GND	2Y	8

7420 (2 4-In NAND)

1	1A	V_{CC}	14
2	1B	2D	13
3	NC	2C	12
4	1C	NC	11
5	1D	2B	10
6	1Y	2A	9
7	GND	2Y	8

7421 (2 4-In AND)

1	1A	V_{CC}	14
2	1B	1C	13
3	2A	1Y	12
4	2B	3C	11
5	2C	3B	10
6	2Y	3A	9
7	GND	3Y	8

7427 (3 3-In NOR)

1	A	V_{CC}	14
2	B	NC	13
3	C	H	12
4	D	G	11
5	E	NC	10
6	F	NC	9
7	GND	Y	8

7430 (8-In NAND)

1	1A	V_{CC}	14
2	1B	4B	13
3	1Y	4A	12
4	2A	4Y	11
5	2B	3B	10
6	2Y	3A	9
7	GND	3Y	8

7432 (4 2-In OR)

1	1A	V_{CC}	14
2	1B	4B	13
3	1Y	4A	12
4	2A	4Y	11
5	2B	3B	10
6	2Y	3A	9
7	GND	3Y	8

7486 (4 2-In EXOR)

1	A	V_{CC}	15
2	B	M	15
3	C	L	14
4	D	K	13
5	E	J	12
6	F	I	11
7	G	H	10
8	GND	Y	9

74133 (13-In NAND)

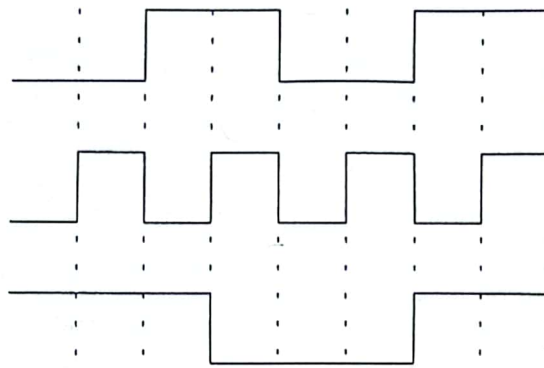
1	1A	V_{CC}	14
2	1B	4B	13
3	1Y	4A	12
4	2Y	4Y	11
5	2A	3Y	10
6	2B	3B	9
7	GND	3A	8

74266 (4 2-In EXNOR)

Figure 4.14 Pin connection of some TTL logic gates.

Exercises

- 4.1 Show that the dual of the EXOR is EXNOR.
- 4.2 By substituting the Boolean function, show that
- The EXOR and EXNOR operations are commutative and associative.
 - The NOR and NAND operators are not distributive.
- 4.3 A *majority* gate is a digital circuit whose output is equal to 1 if the majority of the inputs are 1s. The output is 0 otherwise. By means of a truth table, find the Boolean function implemented by a 3-input majority gate.
- 4.4 Draw the output wave form, if the input waveforms are as given below, for
- 3-input AND gate, (b) 3-input OR gate, (c) 3-input NAND gate, (d) 3-input NOR gate, (e) 3-input EXOR gate, and (f) 3-input EXNOR gate.



- 4.5 How many different ways do we have to implement the NOT operation in logic circuit?
- 4.6 How many different ways do we have to implement AND operation in a logic circuit?
- 4.7 How many different ways do we have to implement OR operation in a logic circuit?
- 4.8 How many alternate symbols do we have to use for NAND, NOR, and NOT gates?
- 4.9 How many different ways do we have to implement EXOR and EXNOR gates?
- 4.10 Show that
- A positive-logic AND gate is a negative-logic OR gate, and vice versa.
 - A positive-logic EXOR gate is a negative-logic EXNOR gate, and vice versa.
 - A positive-logic NAND gate is a negative-logic NOR gate, and vice versa.
- 4.11 Given the Boolean function
- $$F = xy + x'y' + y'z$$
- Implement it with AND, OR, and NOT gates.
 - Implement it with only OR and NOT gates.
 - Implement it with only AND and NOT gates.

Further Reading

1. *The TTL Data Book*. Dallas, Texas Instruments, 1988.
2. Tsutomu Sasao (editor), *Logic Synthesis and Optimization*, Kluwer Academic Publishers, 1993.
3. *Microprocessor Data Hand Book*, Revised Ed. BPB Publications, India, 2000.
4. M. Morris Mano, *Digital Logic and Computer Design*, Prentice-Hall, Inc., 2000.
5. John P. Uyemura, *Digital Systems Design: An Integrated Approach*, Brooks/Cole Publishing Company, 2000.
6. D. C. Green, *Digital Electronics*, 5th ed., Pearson Education, 2001.
7. Ronald J. Tocci and Neal S. Widmer, *Digital Systems: Principles and Applications*, 8th ed., Prentice-Hall, Inc., 2001.
8. V. K. Jain, *Switching Theory and Digital Electronics*, Khanna Publishers, India, 2002.

